Федеральное государственное автономное образовательное учреждение
высшего образования

**Национальный исследовательский ядерный университет
«МИФИ»**

# ЭВОЛЮЦИОННОЕ ПРОГРАММИРОВАНИЕ И ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

## Лабораторная работа № 2

«Генетический алгоритм для дискретной задачи с ограничениями»

Вариант 17

Выполнил: Евдокимов Тимофей Андреевич
Группа: Б24-527

Москва — 2026

# Содержание

**1 Цель работы** **2**

**2 Постановка задачи** **2**

**3 Исходные данные** **2**

**4 Представление и генетические операторы** **3**

**5 Обработка ограничений** **3**

**6 Примеры допустимых и недопустимых решений** **4**

**7 Организация эксперимента** **4**

**8 Результаты** **5**

**9 Лучшее найденное распределение** **6**

**10 Воспроизводимость** **7**

**11 Вывод** **7**

# 1 Цель работы

Реализовать генетический алгоритм для дискретной задачи распределения вычислительных задач по процессорам с учётом ограничений. Исследовать два способа обработки ограничений и два оператора мутации, сопоставить результаты с базовым случайным поиском.

# 2 Постановка задачи

Рассматриваются 30 независимых задач с целочисленными длительностями $t_i$ и пять процессоров. Решение задаётся вектором назначений $x = (x_1, \ldots, x_{30})$. Значение $x_i \in \{0, 1, 2, 3, 4\}$ задаёт процессор для задачи $i$. Загрузка процессора $p$ определяется выражением

$$L_p(x) = \sum_{i : x_i = p} t_i. \tag{1}$$

Целевая функция — максимальная загрузка:

$$\min_x C_{\max}(x), \qquad C_{\max}(x) = \max_{p \in \{0,1,2,3,4\}} L_p(x). \tag{2}$$

Установлены ограничения: $L_p \leq 67$ для каждого процессора; задача 1 должна выполняться процессором 0, задача 8 — процессором 2, задача 20 — процессором 4. Каждая задача назначается ровно одному процессору, что гарантируется кодировкой.

# 3 Исходные данные

Для генерации длительностей используется модуль `random` языка Python. Начальное состояние генератора: 17026.

Длительности выбираются вызовом `randint(3,15)`. Суммарная длительность составляет 247 единиц. Ограничение загрузки определено по формуле

$$L_{\text{limit}} = \left\lceil \frac{247}{5} \cdot 1.35 \right\rceil = 67. \tag{3}$$

Коэффициент запаса и обязательные назначения выбраны в рамках модели, поскольку индивидуальное задание не фиксирует их численно. Полный набор данных и параметров записан в `results/instance.json`.

# 4 Представление и генетические операторы

Генотип — вектор из 30 целочисленных номеров процессоров. Фенотип — распределение задач по пяти процессорам и соответствующие загрузки. Декодирование состоит в группировке задач по значениям генов и суммировании их длительностей.

Применяются турнирная селекция с размером турнира 3, равномерный кроссинговер с вероятностью $P_c = 0.9$ и элитизм с сохранением двух лучших особей. Размер популяции равен 80.

Сравниваются два оператора мутации с вероятностью применения $P_m = 0.3$:

- **move**: переназначение одной задачи другому процессору;
- **swap**: обмен назначениями двух задач.

Оба оператора сохраняют корректность целочисленного кодирования, но могут нарушить ограничения на загрузку и обязательные назначения.

# 5 Обработка ограничений

Исследуются две стратегии.

**Восстановление допустимости (repair).** Сначала исправляются обязательные назначения, затем задачи переносятся с перегруженных процессоров на менее загруженные. Если локальное исправление не удаётся, выполняется жадное построение нового допустимого назначения с учётом фиксированных задач.

**Штрафная функция (penalty).** Недопустимые решения допускаются к оцениванию, но получают штраф:

$$F(x) = C_{\max}(x) + 20 \sum_{p=0}^{4} \max(0, L_p - 67) + 1000 N_{\text{fixed}}, \tag{4}$$

где $N_{\text{fixed}}$ — число нарушений обязательных назначений. В качестве итогового ответа выбирается только допустимое решение.

# 6 Примеры допустимых и недопустимых решений

В файле `results/examples.csv` сохранены четыре примера:

- `valid_1`: допустимое распределение, $C_{\max} = 64$;
- `valid_2`: допустимое распределение, $C_{\max} = 58$;
- `invalid_1`: все задачи на процессоре 0, $C_{\max} = 247$, суммарное превышение загрузки 180 и два нарушения обязательных назначений;
- `invalid_2`: $C_{\max} = 64$, нарушено одно обязательное назначение.

Таким образом, проверяются как ограничения вместимости, так и фиксированные назначения.

# 7 Организация эксперимента

Для каждого метода выполнено 20 независимых запусков. Бюджет одного запуска — 8000 оценённых кандидатов, включая начальную популяцию. Сравниваются:

1. `GA_repair_move` — восстановление допустимости и перенос одной задачи;
2. `GA_penalty_move` — штрафы и перенос одной задачи;
3. `GA_repair_swap` — восстановление допустимости и обмен назначениями;
4. `Random_feasible` — случайная генерация с восстановлением допустимости.

Сравнение первого и второго методов изолирует влияние способа обработки ограничений, первого и третьего — влияние оператора мутации. Все методы имеют одинаковый бюджет по числу оценок, но не обязательно по машинному времени. Для генетических алгоритмов используются seed 10000–10019, для случайного поиска — 20000–20019.

# 8 Результаты

Таблица 1: Итоговая статистика по 20 независимым запускам

| Метод | Допустимых | Лучшее | Среднее | Медиана | Ст. откл. | Худшее |
|---|---|---|---|---|---|---|
| GA_repair_move | 20/20 | 50 | 50.55 | 51 | 0.5104 | 51 |
| GA_penalty_move | 20/20 | 50 | 50.70 | 51 | 0.4702 | 51 |
| GA_repair_swap | 20/20 | 50 | 50.25 | 50 | 0.5501 | 52 |
| Random_feasible | 20/20 | 50 | 51.30 | 51 | 0.6569 | 52 |

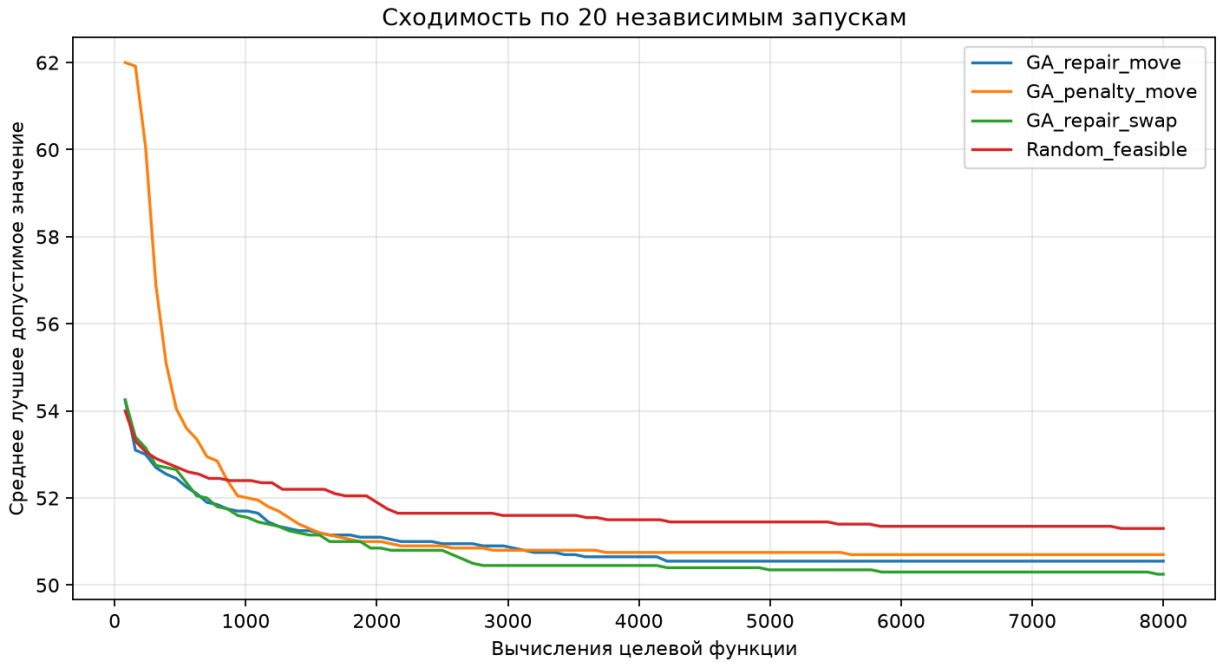



Рис. 1: Сходимость методов по среднему лучшему допустимому значению за 20 запусков




Рис. 2: Распределение лучших результатов отдельных запусков

Наименьшее среднее значение (50.25) показал `GA_repair_swap`. Все методы хотя бы в одном запуске достигли значения 50. Поскольку суммарная длительность задач равна 247, для любого распределения справедлива нижняя граница

$$C_{\max} \geq \left\lceil \frac{247}{5} \right\rceil = 50. \tag{5}$$

Следовательно, найденное допустимое распределение с $C_{\max} = 50$ является глобально оптимальным для рассматриваемого экземпляра задачи. Различия средних результатов невелики и не доказывают универсального превосходства какого-либо оператора.

# 9 Лучшее найденное распределение

В таблице 2 приведено одно из найденных оптимальных допустимых решений. Номера задач указаны с единицы. Все загрузки не превышают установленного ограничения 67.

Таблица 2: Назначение задач по процессорам при $C_{\max} = 50$

| Процессор | Номера назначенных задач | Загрузка | Предел |
|---|---|---|---|
| 0 | 1, 2, 7, 9, 18, 27, 28 | 50 | 67 |
| 1 | 4, 6, 12, 16, 25 | 50 | 67 |
| 2 | 8, 13, 22, 24, 26, 30 | 47 | 67 |
| 3 | 5, 10, 11, 15, 21, 23 | 50 | 67 |
| 4 | 3, 14, 17, 19, 20, 29 | 50 | 67 |

# 10 Воспроизводимость

Реализация находится в `main.py`. Файл `README.md` содержит инструкцию запуска. В каталоге `results/` сохранены исходные данные (`instance.json`), примеры допустимости (`examples.csv`), результаты отдельных запусков (`runs.csv`), сводная статистика (`summary.csv`), лучшее распределение и графики.

Лучшее решение сохранено в файле `best_solution.json`. Все эксперименты воспроизводятся при заданных начальных состояниях генератора случайных чисел.

# 11 Вывод

Реализован генетический алгоритм для дискретного распределения 30 задач по пяти процессорам. Проверены две стратегии обработки ограничений и две специализированные мутации, проведено сравнение со случайным допустимым поиском на одинаковом бюджете оценок. На исследованном наборе данных получено оптимальное значение максимальной загрузки 50, совпадающее с теоретической нижней границей.